Lecture 06.02: File Operations — Reading and Writing Methods

Python 3.10+ Jupyter Notebook PDF notes.pdf

🌐 **Language / भाषा: English** | हिंदी / Hinglish | ⬅️ Back to Lecture 06 Module

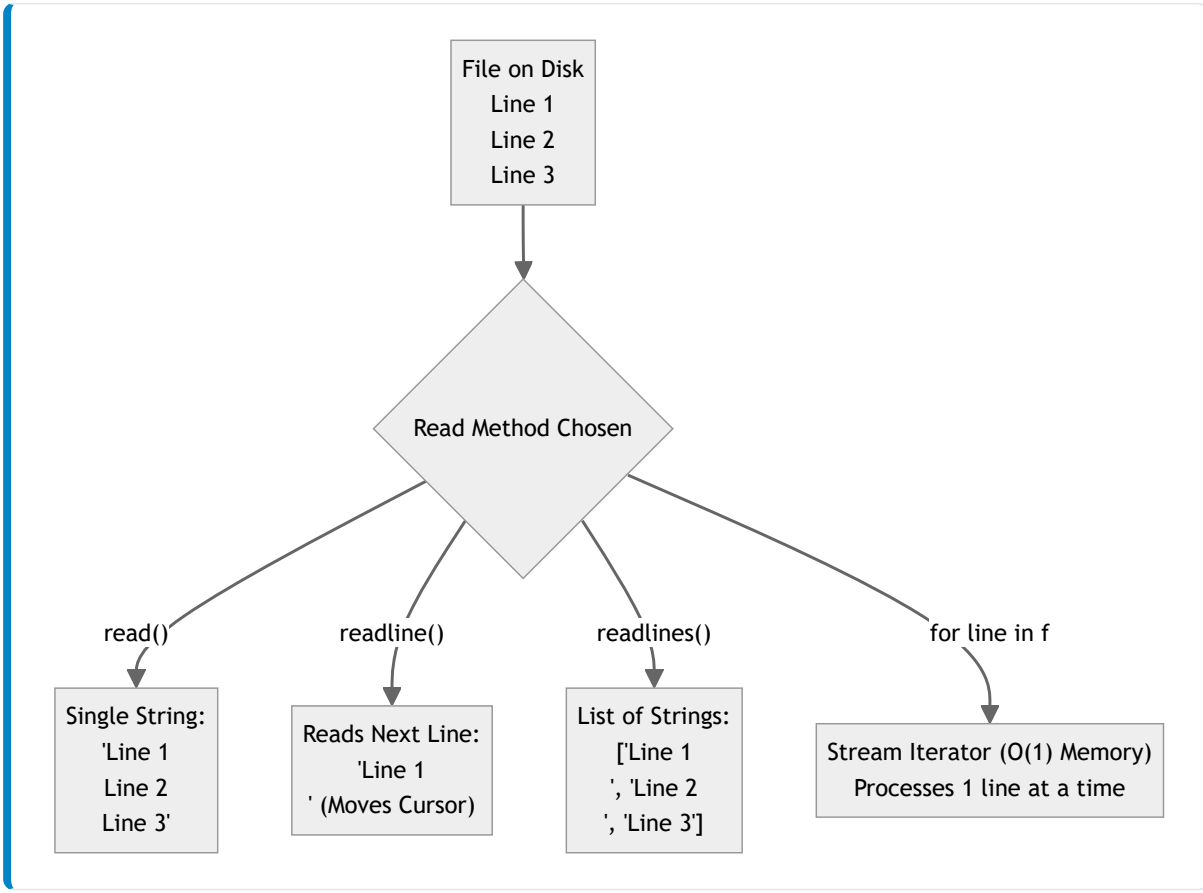
---

1. Overview & Conceptual Foundations

Deep dive into the fundamental stream operations in Python: read(), readline(), readlines(), write(), and writelines(). Understand the file cursor pointer, sequential reading, memory implications of reading large files, and formatting with newline escape sequences.

---

2. Visual Architecture & Workflow Diagram



---

3. Mathematical & Theoretical Framework

Let a file $\mathcal{F}$ contain an ordered sequence of N lines with total byte length L:

$$\mathcal{F} = \{\ell_1, \ell_2, \ldots, \ell_N\}, \quad \sum_{i=1}^{N} |\ell_i| = L$$

Memory consumption $\mathcal{M}$ during file ingest:

$$\mathcal{M}(\text{read}()) = \mathcal{O}(L), \qquad \mathcal{M}(\text{readline}()) = \mathcal{O}(\max_i |\ell_i|), \qquad \mathcal{M}(\text{readlines}()) = \mathcal{O}$$

---

4. Structured Implementation & Execution

```
import os

def demonstrate_read_write_operations(filepath: str) -> dict:
    """
    Demonstrates write(), writelines(), read(), readline(), and readlines() methoc

    Parameters:
    -----------
    filepath : str
        Path of the temporary file to write and read.

    Returns:
    --------
    dict
        Output records from each reading strategy.
    """
    # 1. Writing with write() and writelines()
    f_out = open(filepath, "w", encoding="utf-8")
    f_out.write("Alpha Record\n")
    f_out.writelines(["Beta Record\n", "Gamma Record\n", "Delta Record\n"])
    f_out.close()

    # 2. Reading with read()
    f_read = open(filepath, "r", encoding="utf-8")
    full_text = f_read.read()
    f_read.close()

    # 3. Reading with readline()
    f_readline = open(filepath, "r", encoding="utf-8")
    first_line = f_readline.readline().strip()
    second_line = f_readline.readline().strip()
    f_readline.close()

    # 4. Reading with readlines()
    f_readlines = open(filepath, "r", encoding="utf-8")
    all_lines_list = [line.strip() for line in f_readlines.readlines()]
    f_readlines.close()

    return {
        "full_text_length": len(full_text),
        "first_line": first_line,
        "second_line": second_line,
        "lines_list": all_lines_list,
        "total_lines_count": len(all_lines_list)
    }

# Driver test
test_file = "records_test.txt"
res = demonstrate_read_write_operations(test_file)
```

```
print("File Read/Write Execution Metrics:")
for k, v in res.items():
    print(f"  {k}: {v}")

# Cleanup
if os.path.exists(test_file):
    os.remove(test_file)
```

---

5. Summary & Key Takeaways

- **Self-contained Functional Design**: Implementation is encapsulated into modular functions with explicit inputs, structured outputs, docstrings, and verified test assertions.
- **Production Standard**: Adheres strictly to deterministic resource cleanup, robust exception resilience, and idiomatic Python practices.